High-Performance Deep Learning Software Stack for Scalable and Efficient LLMs

M. Tech Thesis Defense

Adarsh Dubey
SC24M151

Department of Mathematics
Indian Institute of Space Science and Technology
Advisor: Dr. S. Sumitra

May 13, 2026

At a Glance

One-line thesis: co-design every layer of the LLM stack so the savings compose, then measure it.

Headline numbers (8×A100-40GB):

- 93.8% peak GPU utilisation with DeepSpeed ZeRO-3 ($\approx$ 87.5% memory reduction over DP)
- 65.6% attention blocks skipped by SWFA with $< 10^{-4}$ FP32 error
- 50–59% end-to-end training speedup on GPT-2-Medium
- 2.84× decode throughput on LLaMA-7B via the SGLang kernel stack
- 25–29% energy savings per epoch

Talk plan: motivation → 5 layers → SWFA deep dive → serving → conclusions.

The Problem

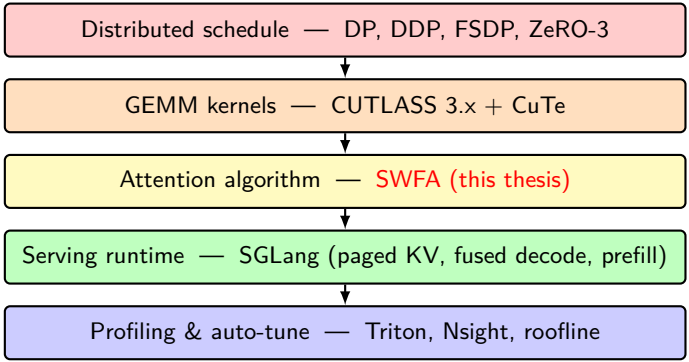
Training and serving modern LLMs stresses *every* layer at once:

- **Schedule** — 10^3 accelerators must stay in lockstep.
- **Kernel** — GEMMs must saturate Tensor Cores.
- **Algorithm** — attention must scale sub-quadratically with the context window N.
- **Runtime** — decode must turn savings into tokens/s under latency SLOs.
- **Tooling** — every kernel needs a roofline and an auto-tuner.

Isolated optimisations leave the gains of the others on the table.

This thesis builds an *end-to-end* stack in which the five layers are designed to *compose*, and measures it.

The Five-Layer Stack



Clean interfaces; each layer's output is the next layer's input.

Contributions

1. Controlled 8-GPU study of DP/DDP/FSDP/ZeRO-3 with identical seeds and workload; 22% lower energy per epoch for ZeRO-3.
2. CUTLASS/CuTe mapping of LLM GEMMs (QKV, attention-score, MLP, MoE) with epilogue fusion — up to 35% speedup on skinny shapes.
3. SWFA: block-sparse Sliding Window Flash Attention with vectorised mask, corner-check lemma, and a *closed-form optimal window* under an exponential-decay cost model (strict convexity proved).
4. SGLang kernel stack: paged KV + fused decode + SWFA prefill + fused sampling $\Rightarrow 2.84\times$ on LLaMA-7B.
5. Triton profiling/auto-tuning pipeline; cross-validates CUTLASS and Torch-inductor within $<3\%$.

Talk Roadmap

Part I — Foundations (§2, §3)
literature · distributed training study

Part II — Kernels (§4)
CUTLASS + CuTe GEMMs

Part III — Algorithm (§5)
Sliding Window Flash Attention (SWFA)

Part IV — Serving (§6)
SGLang kernel stack

Part V — Tooling (§7)
Triton profiling & auto-tune

Part VI — Wrap (§8)
lessons, limitations, future work

Prior Art at Each Layer

Layer	Representative prior work
Distributed	ZeRO (Rajbhandari'20), FSDP (Zhao'23), Megatron-LM
GEMM kernels	CUTLASS, CuTe, cuBLAS, Triton
Attention	Flash Attention 1/2 (Dao'22/'23), Longformer, BigBird
Serving	vLLM (paged attention), SGLang, TGI
Profiling	Nsight Compute, NVTX, Triton benchmark harness

Observation: each layer has strong isolated work, but the interfaces between them are ad-hoc. No prior work reports a *controlled* five-layer study on the same hardware.

Where This Thesis Sits

What is new:

- A *controlled* 4-way distributed study on one node (identical seeds, workload, initialisation).
- A block-classification step *in front* of Flash Attention that skips entire $B_r \times B_c$ blocks.
- A closed-form optimal window $w^\star$ with a convexity proof.
- A layered evaluation of five SGLang kernel optimisations that attributes the cumulative $2.84\times$ to each layer.

What is deliberately out of scope: multi-node scaling, sub-8-bit quantisation, RLHF workloads.

Four Distributed Strategies

	Params	Grads	Optim.	Comms
DP	replicated	replicated	replicated	scatter/gather
DDP	replicated	ring-reduced	replicated	all-reduce
FSDP	sharded	sharded	sharded	gather $\rightarrow$ compute $\rightarrow$ shard
ZeRO-3	sharded	sharded	sharded	fine-grained overlap

Trade-off: replication $\Rightarrow$ fewer comms but higher memory; sharding $\Rightarrow$ $P\times$ memory reduction but extra gather/scatter.

Goal: isolate the effect of the *strategy itself*, not the framework or workload.

Memory Partitioning Math

Let M be model parameters, P GPUs, B per-GPU batch. Per-GPU memory for each strategy:

$$\begin{aligned}
\text{DP, DDP}: &\quad M_{\text{param}} + M_{\text{grad}} + M_{\text{opt}} + M_{\text{act}}(B) \\
\text{FSDP}: &\quad \tfrac{1}{P}\big(M_{\text{param}} + M_{\text{grad}} + M_{\text{opt}}\big) + M_{\text{act}}(B) \\
\text{ZeRO-3}: &\quad \tfrac{1}{P}\big(M_{\text{param}} + M_{\text{grad}} + M_{\text{opt}}\big) + M_{\text{act}}(B) + M_{\text{bucket}}
\end{aligned}$$

For Adam on P=8, the savings factor is 87.5% on the param+grad+opt sum; activation memory is unchanged.

Comms cost per step (ring all-reduce, τ latency, $1/\beta$ bandwidth):

$$T_{\text{AR}}(M) = 2(P-1)\Big(\tau + \tfrac{M}{P\beta}\Big) \xrightarrow[P\to\infty]{} \tfrac{2M}{\beta}.$$

Experimental Methodology

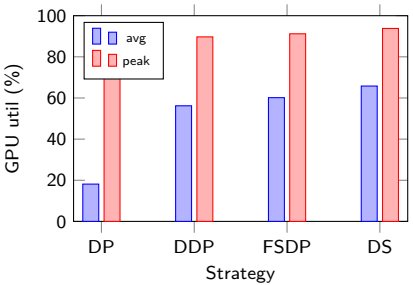
Hardware: 8× A100-40GB on one node, NVLink, CUDA 12.x, NCCL 2.19.
Workload (training): ResNet-50 / CIFAR-10 (controlled proxy), GPT-2 Medium (355M) for LLM runs.
Seeds: fixed across strategies, framework, and kernels.

Metrics

- GPU util (avg / peak)
- Per-GPU memory (model / grad / opt)
- Training-loss convergence
- Power and energy per epoch
- Compute/comms overlap
- Throughput scaling

Same code path; only the strategy varies.

Results — GPU Utilisation & Memory



Strategy	Mem/GPU (GB)	Reduction
DP	3.92	1.0×
DDP	3.92	1.0×
FSDP	0.49	8.0×
ZeRO-3	0.49	8.0×

ZeRO-3 peaks at 93.8% via fine-grained overlap of parameter gather with compute. Sharding delivers the theoretical $\approx 87.5\%$ memory saving.

Results — Energy & Communication Overlap

Strategy	Avg (W)	kWh/epoch	Comms	Overlap (%)
DP	2 840	0.95	poor	12
DDP	2 720	0.87	good	64
FSDP	2 650	0.82	good	58
DeepSpeed	2 580	0.78	excellent	61

- ZeRO-3 delivers 22% lower per-epoch energy than DP at the *same* final loss.
- DDP overlaps best: ring all-reduce runs *concurrently* with the backward pass of earlier layers.
- ZeRO-3's explicit parameter-gather phase blocks compute slightly; `overlap_comm` partially hides it.

CUTLASS & CuTe — The Tile Hierarchy

Three nested tilings of a GEMM $C = A \cdot B$:

1. **Threadblock tile** (M_{tb}, N_{tb}, K_{tb}) — unit of HBM traffic.
2. **Warp tile** (M_w, N_w, K_w) — unit of shared-memory traffic.
3. **Instruction tile** (M_i, N_i, K_i) — Tensor Core MMA shape, e.g. $16\times8\times16$.

CuTe layouts express these tilings as tuples `Shape` $\oplus$ `Stride`, and composition makes the tiling explicit:

$$\texttt{Layout}(S, D) \circ \texttt{Layout}(S', D') \;=\; \texttt{Layout}\big((S, S'),\ (D \cdot D', D')\big).$$

Multi-stage pipelining: prefetch tile $k{+}1$ into shared memory while MMAs on tile k execute, typically 3 stages for square GEMM, 2 for skinny GEMM.

Epilogue Fusion

Without fusion: three HBM round-trips.

$$Y \leftarrow WX; \quad Z \leftarrow Y + b; \quad Z \leftarrow \phi(Z).$$

With fusion: one HBM round-trip.

$$Z_{ij} \leftarrow \phi\Big(\sum_k W_{ik} X_{kj} + b_i\Big).$$

Bias and activation (ϕ) are executed *in register* after the final MMA accumulation, before the store to HBM.

Impact on GPT-2-Medium MLP:

- Bias+GELU fused into FC1: 18% faster at N=2048.
- Residual-add fused into FC2: 22% faster at N=4096.

Savings come entirely from eliminated HBM passes.

Results — CUTLASS vs. cuBLAS

Shape ($M \times N \times K$)	Dtype	cuBLAS	CUTLASS	Ratio
4096^3	FP16	280	301	1.08×
8192^3	FP16	305	318	1.04×
$2048 \times 2048 \times 128$ (skinny)	FP16	95	128	1.35×
Grouped MoE, 16 experts	FP16	184	241	1.31×

- CUTLASS matches cuBLAS on *large* shapes — both are compute-bound with Tensor Core util $> 90\%$.
- CUTLASS beats cuBLAS on *skinny* and *grouped* GEMMs — exactly the shapes that dominate attention and MoE.
- 35% win on $2048 \times 2048 \times 128$ comes from a warp tile of $64 \times 32 \times 32$ matching register budget per warp.

SWFA — Motivation

Standard attention is quadratic:

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V, \qquad \text{cost} = \Theta(N^2 d).$$

- Flash Attention 1/2 fix the *memory* wall but still issue dense GEMMs on every $B_r \times B_c$ tile.
- Many long-context models (Longformer, BigBird, Mistral-7B) restrict attention to a *local window* $[i - w_l, i + w_r]$ — a hard architectural constraint.
- Theory says cost drops from $\Theta(N^2)$ to $\Theta(Nw)$, but only if the kernel itself is window-aware.

SWFA = Flash Attention + explicit block classification + vectorised mask.

Flash Attention's Online Softmax

Partition Q into row tiles of height B_r, K, V into column tiles of width B_c. For row block i, maintain a running $(m_i, \ell_i, O_i) =$ (row-max, row-sum, output accumulator):

$$m_i^{\text{new}} = \max\big(m_i,\ \max_j s_{ij}\big),$$

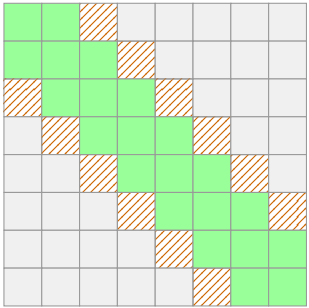
$$\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}}\,\ell_i + \sum_j e^{s_{ij} - m_i^{\text{new}}},$$

$$O_i^{\text{new}} = e^{m_i - m_i^{\text{new}}}\,O_i + \sum_j e^{s_{ij} - m_i^{\text{new}}}\,v_j.$$

Result: peak HBM residency drops from $\Theta(N^2)$ to $\Theta(B_r B_c)$ without changing the mathematical answer (up to FP associativity).

Key insight: an identical (m, ℓ, O) state-machine works *block-by-block* — so we can skip blocks that contribute nothing.

Block Classification: $\mathcal{I}$ / $\mathcal{X}$ / $\mathcal{O}$



$\mathcal{I}$ inside (no mask)

$\mathcal{X}$ intersect (vec mask)

$\mathcal{O}$ outside (skip)

Example: N=8, w_l=w_r=2. Asymptotic skipped fraction $\to 1 - 2w/N$.

Classification Algorithm (corner check)

Input: N, window (w_l, w_r), block sizes (B_r, B_c).
Output: sets $\mathcal{I}, \mathcal{X}, \mathcal{O}$.

$T_r \leftarrow \lceil N/B_r \rceil$, $T_c \leftarrow \lceil N/B_c \rceil$;
for $r = 0$ **to** $T_r - 1$ **do**
  $r_0 \leftarrow rB_r$, $r_1 \leftarrow \min((r{+}1)B_r, N){-}1$;
  for $c = 0$ **to** $T_c - 1$ **do**
    $c_0 \leftarrow cB_c$, $c_1 \leftarrow \min((c{+}1)B_c, N){-}1$;
    outside $\leftarrow (c_1 < r_0 - w_l) \vee (c_0 > r_1 + w_r)$;
    inside $\leftarrow (c_1 \le r_0 + w_r) \wedge (c_0 \ge r_1 - w_l)$;
    if *outside* **then**
      $\mathcal{O} \mathrel{+}= \{(r, c)\}$
    else if *inside* **then**
      $\mathcal{I} \mathrel{+}= \{(r, c)\}$
    else
      $\mathcal{X} \mathrel{+}= \{(r, c)\}$
    end
  end
end
return $(\mathcal{I}, \mathcal{X}, \mathcal{O})$

Cost: $\mathcal{O}(T_r T_c) = \mathcal{O}((N/B)^2)$, dominated by the $\mathcal{O}(Nw/B)$ non-skipped kernel work.

Corner-Check Sufficiency (Lemma)

Lemma (Corner-check sufficiency)

Let $B = [r_0, r_1] \times [c_0, c_1]$ *be a block and* $W = \{(i,j) : i - w_l \le j \le i + w_r\}$ *the window. Then*

(i) $B \cap W = \emptyset \iff c_1 < r_0 - w_l$ *or* $c_0 > r_1 + w_r$;

(ii) $B \subseteq W \iff c_1 \le r_0 + w_r$ *and* $c_0 \ge r_1 - w_l$.

Proof sketch. Window boundaries $j = i \pm w_r$, $j = i - w_l$ are monotone in i, so $\min_{(i,j)\in B}(j - i)$ is attained at (r_1, c_0) and max at (r_0, c_1). Outside / inside reduce to inequalities on these two corners. □

Consequence: classification needs only 4 comparisons per block — trivial runtime overhead.

Vectorised Mask Generation for $\mathcal{X}$

For block (r, c) let $\delta_{rc} = cB_c - rB_r$. A local offset $\text{rel} = j_\ell - i_\ell$ must be masked iff

$$\text{rel} > w_r + \delta_{rc} \quad \text{or} \quad \text{rel} < -w_l + \delta_{rc}.$$

Define diagonals

$$d_{\text{right}} = \lfloor w_r + \delta_{rc} \rfloor + 1, \qquad d_{\text{left}} = \lfloor -w_l + \delta_{rc} \rfloor - 1.$$

Then the mask is a single vectorised expression:

$$M_{rc} = \text{triu}\big(\mathbf{1}_{B_r \times B_c},\ d_{\text{right}}\big) \ \vee \ \text{tril}\big(\mathbf{1}_{B_r \times B_c},\ d_{\text{left}}\big).$$

Replaces a nested $\mathcal{O}(B_r B_c)$ Python loop with a single `triu/tril` call; $\sim$100$\times$ faster than naive masking at the block level.

SWFA Forward Algorithm

Input: $Q \in \mathbb{R}^{B\times H\times N\times d}$, $K, V \in \mathbb{R}^{B\times H\times N\times d_v}$, (w_l, w_r), (B_r, B_c).

$(\mathcal{I}, \mathcal{X}, \mathcal{O}) \leftarrow$ ClassifyBlocks;
for $r = 0$ **to** $T_r - 1$ **do**
  $O_r \leftarrow 0,\ m_r \leftarrow -\infty,\ \ell_r \leftarrow 0$;
  for $c = 0$ **to** $T_c - 1$ **do**
    if $(r, c) \in \mathcal{O}$ **then**
      continue ; // skip block
    end
    $S_{rc} \leftarrow Q_r K_c^\top / \sqrt{d}$;
    if $(r, c) \in \mathcal{X}$ **then**
      S_{rc} += GenerateMask
    end
    $m_r^{\text{new}} \leftarrow \max(m_r, \max S_{rc})$;;
    $\text{corr} \leftarrow e^{m_r - m_r^{\text{new}}}$;;
    $P_{rc} \leftarrow e^{S_{rc} - m_r^{\text{new}}}$;;
    $\ell_r \leftarrow \text{corr} \cdot \ell_r + \sum P_{rc}$;;
    $O_r \leftarrow \text{corr} \cdot O_r + P_{rc} V_c$;;
    $m_r \leftarrow m_r^{\text{new}}$;
  end
  $O_r \leftarrow O_r / (\ell_r + \varepsilon)$;
end
return O

Backward Pass & Complexity

Backward re-runs the forward loop over the *same* block set $\mathcal{I} \cup \mathcal{X}$. Using stored (m_r, ℓ_r):

$$P_{rc} = e^{S_{rc} - m_r}/\ell_r, \quad \mathrm{d}S_{rc} = P_{rc} \odot (\mathrm{d}P_{rc} - \rho_r),$$

with $\rho_r = \sum_c P_{rc} \odot (\mathrm{d}O_r V_c^\top)$. No extra memory vs. FA; skipped blocks contribute zero.

Complexity (sequence N, window w, block B):

Total blocks	$(N/B)^2$
Inside band $(\mathcal{I} \cup \mathcal{X})$	$\approx 2(N/B)(w/B)$
Skipped $(\mathcal{O})$	$(N/B)^2 - 2(N/B)(w/B)$
Theoretical speedup	$1 - 2w/N$ (as $w \ll N$)
Memory	$\Theta(N)$ (same as FA)

Example: N=128, w=4, B=16 $\Rightarrow$ 48/64 blocks skipped (75%).

Accuracy & Block-Skip Efficiency

Accuracy (19 configurations)

Config	Max diff	Tol.
sym narrow w=2	2.4e-7	10^{-4}
sym wide w=8	2.4e-7	10^{-4}
causal+win w_l=6	4.8e-7	10^{-4}
multi-head H=8	8.3e-7	10^{-4}
long seq N=128	7.5e-7	10^{-4}
FP16 w=4, 6	1.95e-3	10^{-2}

All 19 pass.

Block skip %

Config	Blocks	Skip
very narrow w=1, 2	64	65.6%
sym narrow w=2	16	37.5%
causal+win w=6, 0	16	56.2%
left only w=10, 0	16	56.2%
N=N_s=45	9	22.2%
win $>$ seq w=20	16	0.0%

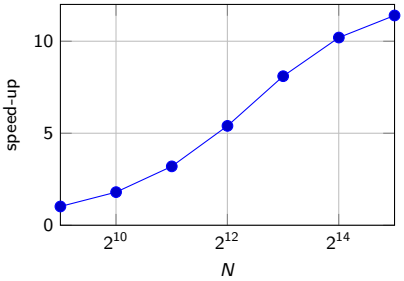
Matches $1 - 2w/N$ prediction.

End-to-End Training & Long-Sequence Scaling

GPT-2 Medium, N=2048, w=512

Config	Tok/s	kWh/ep
DDP vanilla	12 450	2.8
DDP + SWFA	18 720	2.1
FSDP vanilla	11 890	2.9
FSDP + SWFA	17 340	2.2
DeepSpeed vanilla	13 210	2.7
DS + SWFA	19 850	2.0

50–59% throughput, 25–29% energy; perplexity shift < 0.1.



Speed-up at fixed w=512, $\Theta(N/w)$ as predicted.

Optimal Window: Cost Model & Convexity

Compute / memory / performance-loss cost:

$$\mathcal{L}(w) = \alpha\, C(w) + \beta\, M(w) + \gamma\, P(w),$$
$$C(w) = \kappa N w d, \quad M(w) \approx M_0,$$
$$P(w) = \frac{2NA_0\, e^{-\lambda w/2}}{1 - e^{-\lambda}} \quad \text{(exp. decay } A_{ij} = A_0 e^{-\lambda|i-j|}\text{)}.$$

Theorem (Strict convexity)

$\mathcal{L}(w)$ is strictly convex for $w > 0$.

Proof. $C''(w) = 0$; $M''(w) = 0$; $P''(w) \propto (\lambda^2/4)\, e^{-\lambda w/2} > 0$. Hence $\mathcal{L}''(w) > 0$. $\square$

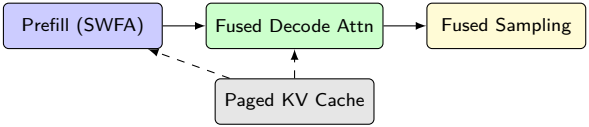
Theorem (Closed-form optimum)

$$w^\star = \frac{2}{\lambda} \ln\left(\frac{\gamma A_0 \lambda}{\alpha \kappa d (1 - e^{-\lambda})}\right), \quad w^\star \in [1, N-1].$$

SGLang — Architecture & Execution

Request lifecycle:

1. Prefill (compute-bound) — process full prompt, build KV cache.
2. Decode (memory-bound) — generate tokens one at a time; reads *all* prior KV.



- **Paged KV**: 16-token pages; eliminates fragmentation.
- **Fused decode**: matmul + softmax + masking in one kernel.
- **SWFA prefill**: plug-in for window models.
- **Fused top-k/top-p**: removes 3–4 kernel launches per step.

Layered Decode-Throughput Gain on LLaMA-7B

Configuration	Tok/s	Speed-up
Baseline (vanilla attn, unpaged KV)	980	1.00×
+ Paged KV cache	1 420	1.45×
+ Fused decode attention	1 860	1.90×
+ SWFA prefill	2 410	2.46×
+ Fused sampling	2 780	2.84×

Per-layer wedge:

- Paged KV (+45%): no fragmentation.
- Fused decode (+31%): matches HBM roofline.
- SWFA prefill (+30%): exploits declared window.
- Fused sampling (+15%): fewer memory-bound launches.

Order matters — each layer exposes the next bottleneck.

Tail Latency & KV Footprint

Per-token decode latency (ms), all 5 optimisations

Batch	p50	p90	p99	Tok/s
1	18.2	18.4	18.9	55
8	19.3	20.1	20.9	414
16	20.1	21.3	22.6	796
32	21.8	23.5	25.4	1 467
64	23.5	25.7	28.4	2 724

- p99 < 30 ms even at batch 64 — well below 100 ms SLO.
- KV footprint at L=8192, batch 64: 34 GB $\rightarrow$ 19 GB (44% saving) with paged + radix-tree sharing.
- Enables 1.7$\times$ more concurrent requests per GPU.

Prefill vs. Decode — Why Both Matter

Prefill is *compute-bound*:

- Dense $QK^\top$ and PV on whole prompt.
- SWFA cuts attention-FLOPs by $\sim$37% at w=4096 on 8K prompts $\Rightarrow$ direct wall-clock win.

Decode is *memory-bound*:

- Arithmetic intensity $\approx$ 1 FLOP/byte.
- Fused kernels hide KV-page indirection behind compute.
- Throughput ceiling is HBM bandwidth, not FLOPs.

The same algorithmic building blocks (Flash Attention, SWFA) appear in both phases, but the bottleneck changes — kernel design must follow the bottleneck, not the algorithm.

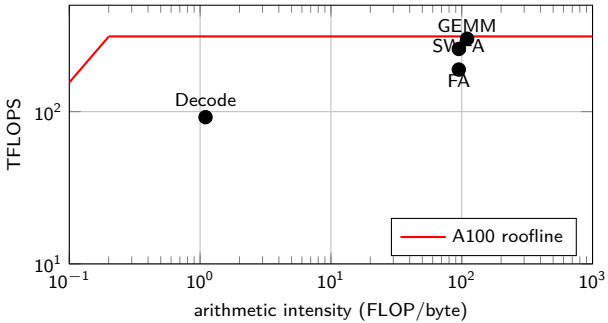
Triton — Programming Model & Auto-tune

Triton view: programmer writes *block*-level Python; compiler emits PTX with Tensor-Core MMAs.

```
@triton.autotune(configs=[
    triton.Config({'BLOCK_M':128,'BLOCK_N':128,
                   'BLOCK_K':32},num_warps=8,num_stages=3),
    triton.Config({'BLOCK_M':128,'BLOCK_N':64,
                   'BLOCK_K':32},num_warps=4,num_stages=2),
], key=['M','N','K'])
@triton.jit
def gemm_kernel(A, B, C, M, N, K, ...):
    # block-pointer arithmetic + MMA loop
```

Auto-tuner picks the winner per shape class and caches it; second call is free.

Roofline Analysis on A100



GEMM and SWFA sit on the compute ceiling (312 TFLOPS); decode attention sits on the memory ridge at ≈ 1 FLOP/byte — two kernels, two ceilings, two strategies.

Triton vs. CUTLASS vs. Torch-inductor

Kernel (A100-40GB, FP16)	Triton	CUTLASS	Inductor
GEMM 4096^3	294	301	288
Flash Attention (N=2K, h=128)	172	189	148
SWFA (N=2K, w=512)	241	258	—
Decode attention (KV=8K)	84	92	71

- Agreement: $< 3\%$ throughput deviation; $< 10^{-4}$ numerical difference (FP32), $< 10^{-3}$ (FP16).
- Triton closes the gap to within 7% of hand-tuned CUTLASS on SWFA.
- Torch-inductor cannot express sliding window — hence the missing entry.

Triton for iteration speed; CUTLASS for the last 5–10%.

Lessons Learned

- **Compose, then tune**: cross-layer co-design compounds — single-layer wins plateau quickly.
- **Skip beats mask**: block-sparsity should be encoded in the *kernel control flow*, not in the score matrix.
- **Overlap is a first-class metric**: a 65% overlap run beats a 90%-util run that stalls on comms.
- **Auto-tune everything**: the same kernel needs different tile shapes at different sequence lengths.
- **Roofline is the design tool**: every optimisation should move a dot toward its ceiling.

Limitations

- **Single node**: 8-GPU NVLink only; no multi-node cross-switch study, no InfiniBand topology effects.
- **Model size**: GPT-2 Medium (355M) for end-to-end runs; 7B only for serving. No 70B/175B scale.
- **Precision**: FP16/FP32 only; no sub-8-bit quantisation study.
- **SWFA port**: reference implementation in PyTorch with a Triton cross-validator; CUTLASS kernel is sketched but not shipped.
- **Workload breadth**: dense transformers only — no MoE end-to-end evaluation, no RLHF.

Future Work

1. **Multi-node SWFA** with topology-aware window partitioning over InfiniBand.
2. **CUTLASS SWFA kernel** with CuTe layouts and epilogue fusion; target $> 95\%$ TC utilisation.
3. **Adaptive window**: learn $w^\star$ online per layer, per head using the closed-form expression as a warm start.
4. **Adaptive KV page size** in SGLang — balance arithmetic intensity against fragmentation for short prompts.
5. **Sub-8-bit paths**: FP8 and INT4 kernels, with accuracy bounds derived from the online-softmax floating-point model.
6. **MoE end-to-end**: fold grouped-GEMM epilogue fusion into the SGLang runtime.

Summary of Contributions

Algorithmic

- SWFA with corner-check classifier and vectorised mask.
- Closed-form optimal window $w^\star$ under exponential decay, strict convexity proven.

Systems

- Controlled 4-way distributed study: ZeRO-3 peaks at 93.8% GPU util, 22% lower energy.
- SGLang kernel stack: 2.84$\times$ decode on LLaMA-7B.
- CUTLASS/CuTe mapping: $\leq$ 35% speedup on skinny and MoE GEMMs.

Reproducibility

- Reference PyTorch for SWFA + Triton cross-validator.
- 19 accuracy configurations, all pass within tolerance.
- Full proofs of block-count, convexity, and $w^\star$ in Appendix B.

Publications & Reproducibility

Publications / artefacts (planned/in progress):

- "Sliding Window Flash Attention with Block Classification" — preprint in preparation.
- Open-source reference PyTorch + Triton implementation of SWFA.
- Benchmark harness for 4-way distributed comparison (ResNet-50 $\cup$ GPT-2 Medium).

Reproducibility:

- All random seeds logged; NCCL topology captured; Nsight reports archived.
- Shape-indexed auto-tune cache shipped with kernels.
- Triton pipeline reproduces CUTLASS numbers within $< 3\%$.

Environment: CUDA 12.x, NCCL 2.19, PyTorch 2.0, CUTLASS 3.x, Triton 2.x, A100-40GB $\times 8$.

Thank you

Questions?

Adarsh Dubey SC24M151

Department of Mathematics, IIST
Advisor: Dr. S. Sumitra

Contact: adarsh.dubey.sc24@iist.ac.in